

```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.P03.Boot
{
    public static class SceneIds
    {
        {
            public const string Boot = "S00_Boot";
            public const string Main = "S03_Main";
        }
        /// <summary>
        /// S00_Boot      Mock → →      T001④
        ///      Boot      UI T001b  Boot      =
        /// </summary>
        public sealed class BootRunner : MonoBehaviour
        {
            IEnumerator Start()
            {
                BootServices.RegisterAll();
                var sequence = new BootSequence(SwitchToMain);
                if (!sequence.RunAll())
                {
                    Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                    yield break;
                }
                yield return null;
            }
            bool SwitchToMain()
            {
                StartCoroutine(LoadMainRoutine());
                return true;
            }
            IEnumerator LoadMainRoutine()
            {
                {
                    var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
                    while (op != null && !op.isDone) yield return null;
                }
            }
        }
    }
}

// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P03.Boot
{
    /// <summary>
```

```

/// Boot      T001
/// ①Logger→② →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦      [BOOT] n/7
/// ⑦      Play =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
/// X006      SY MOCK
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &= Run(1, "Logger", LoggerStep);
        ok &= Run(2, "Time", TimeStep);
        ok &= Run(3, "Config", ConfigStep);
        ok &= Run(4, "Save", SaveStep);
        ok &= Run(5, "ILocale", LocaleStep);
        ok &= Run(6, "Ads.Preload", AdsStep);
        ok &= Run(7, "SwitchMain", () =&gt; _switchScene != null && _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; //      JS sink platform T001c
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 && !time.TimeCheated();
    }
}

```

```

        static bool ConfigStep()
        {
            var config = ServiceLocator.Get<IConfigService>();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock= X009 manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get<ISaveService>();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get<ILocaleService>();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); // +
        }
        static bool AdsStep()
        {
            ServiceLocator.Get<IAdsService>().Preload("boot_warmup");
            return true;
        }
    }
}

// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P03.Boot
{
    /// <summary>
    /// Mock SY MOCK X006 Mock X002a AppID touristappid
    /// T001c WX/TT Channel Editor Boot
    /// </summary>
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet<ITimeService>(out _)) return; //
            ServiceLocator.Register<ITimeService>(new MockTimeService());
            ServiceLocator.Register<IConfigService>(new MockConfigService());
            ServiceLocator.Register<ISaveService>(new MockSaveService());
            ServiceLocator.Register<ILocaleService>(new MockLocaleService());
            ServiceLocator.Register<IAdsService>(new MockAdsService());
            Logger.Info("BOOT", "mock services registered (SY MOCK)");
        }
    }
}

```

```

}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source => TimeSource.Mock;
    public long ClientNowMs() => DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// <summary>SY MOCK X006 TCB serverTime→HTTPS Date → T001b </summary>
    public long ServerNowMs() => ClientNowMs();
    public bool TimeCheated() => Math.Abs(ServerNowMs() - ClientNowMs()) > 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable<T>(string tableId) where T : class
        => throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable<T>(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        => new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) => "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) => PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) => PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// <summary>X006 T001b v1→v2 </summary>
    public SyncResult CloudSync() => new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// <summary>key key T001b cfg_i18n + NUnit </summary>
    public string GetText(string key, params string[] args)
    {
        Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
        return key;
    }
    public FontSet GetFontSet()
        => new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
}
public sealed class MockAdsService : IAdsService
{
    public event Action<AdLifecycle> OnAdEvent;
    public Task<RewardResult> ShowRewarded(string pointId)
        => Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
    public void ShowInterstitial(string pointId) { }
    public void ShowBanner(string pointId) { }
}

```

```

        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}

// MainUI.cs
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
namespace Sy.P03.Newbie
{
    public static class NewbieSceneIds
    {
        {
            public const string Main = "S03_Main";
            public const string Battle = "S03_Battle";
        }
        /// <summary>
        /// S03    V1.4    ·0-20261010-1245
        ///      StartButton      L1
        ///      /          /    /    /
        ///      U362      +      HUD      6
        ///      T5
        /// </summary>
        public sealed class MainUI : MonoBehaviour
        {
            static readonly Color RevRed = new Color(0.757f, 0.153f, 0.176f);
            static readonly Color PaperYellow = new Color(0.929f, 0.871f, 0.706f);
            static readonly Color InkBlack = new Color(0.043f, 0.043f, 0.047f);
            static readonly Color DoomGreen = new Color(0.298f, 0.686f, 0.314f);
            static readonly Color LockedGray = new Color(0.35f, 0.35f, 0.35f, 0.85f);
            Canvas _canvas;
            Sprite _square;
            Text _walletText;
            Text _offlineText;
            Button _offlineButton;
            RectTransform _codexPanel;
            int _lastWallet = -1;
            /// <summary> StartButton </summary>
            public void StartBattle()
            {
                BattleLaunch.StartTutorial();
                SceneManager.LoadScene(NewbieSceneIds.Battle);
            }
            void Awake()
            {
                Application.targetFrameRate = 30;
                Sy.Core.MainArtInjector.Apply(transform, "P03_scene_01", "P03_unit_01");
            }
        }
    }
}

```

```
_canvas = FindObjectOfType<Canvas>();
_square = MakeSquareSprite();
// StartButton
var old = GameObject.Find("StartButton");
if (old != null)
{
    var rt = (RectTransform)old.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = new Vector2(0, 320);
    rt.sizeDelta = new Vector2(560, 130);
    var label = old.GetComponent<Text>();
    if (label != null)
    {
        label.text = "    L1";
        label.fontSize = 52;
    }
    var img = old.GetComponent<Image>();
    if (img != null) img.color = DoomGreen;
}
BuildModeButtons();
BuildWalletLine();
BuildCodexPanel();
SyncWallet(force: true);
}
void BuildModeButtons()
{
    MakeModeButton("    ", new Vector2(0, 160), RevRed, () =>{
        {
            BattleLaunch.StartRandomSiege();
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
    });
    if (TianmingGate.IsEligible())
    {
        MakeModeButton("    ", new Vector2(0, 0), PaperYellow, () =>{
            {
                BattleLaunch.StartTianming();
                SceneManager.LoadScene(NewbieSceneIds.Battle);
            }
        });
    }
    MakeModeButton("    ", new Vector2(-300, -180), InkBlack, ToggleCodex);
    MakeModeButton("    ", new Vector2(300, -180), InkBlack, () =>{
        {
            var wallet = BannerWallet.Load();
            int got = wallet.TryExchange(Random.Range(0, 99));
            SyncWallet(force: true);
            ShowToast(got > 0
                ? "    " + GeneralCodexById(got)?.Name + "    "
                : wallet.CanExchange ? "    " : "    60    ");
        }
    });
}
```

```
void BuildWalletLine()
{
    _walletText = MakeText(_canvas.transform, "    0", 44, new Vector2(0, -320), TextAnchor.MiddleCenter, 500, 60);
    _walletText.color = PaperYellow;
    _offlineButton = MakeModeButtonCore("    ·    ", new Vector2(0, -400), DoomGreen, TryClaimOffline);
    _offlineText = _walletText;    //
}

void TryClaimOffline()
{
    long now = System.DateTimeOffset.Now.ToUnixTimeMilliseconds();
    var claim = OfflinePay.OfflineClaim.Open(now);
    if (!claim.Claimable)
    {
        ShowToast("    1    ");
        return;
    }
    int got = claim.TryCollect(false);
    if (got > 0)
    {
        SyncWallet(force: true);
        ShowToast("    +" + got);
    }
}

void SyncWallet(bool force)
{
    int balance = BannerWallet.Load().Balance;
    if (!force && balance == _lastWallet) return;
    _lastWallet = balance;
    _walletText.text = "    " + balance + (balance >= BannerWallet.ExchangeCost ? "    " : "");
}

// ——    ——

void BuildCodexPanel()
{
    _codexPanel = MakeOverlayPanel();
    MakeText(_codexPanel, "    ·    ", 72, new Vector2(0, 760), TextAnchor.MiddleCenter, 900, 90).color = Paper
    var generals = GeneralCodex.All;
    for (int i = 0; i < generals.Count; i++)
    {
        int row = i / 2, col = i % 2;
        var g = generals[i];
        bool unlocked = GeneralCodex.IsUnlocked(g.Id);
        var card = new GameObject("Codex_" + g.Id, typeof(RectTransform), typeof(Image));
        card.transform.SetParent(_codexPanel, false);
        var rt = (RectTransform)card.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2(-300 + col * 600, 520 - row * 230);
        rt.sizeDelta = new Vector2(560, 200);
        var img = card.GetComponent<Image>();
        img.sprite = _square;
        img.color = unlocked ? (g.Tier == GeneralCodex.Tier.Gold ? PaperYellow
```

```

        : g.Tier == GeneralCodex.Tier.Purple ? RevRed : DoomGreen) : LockedGray;
    MakeText(card.transform, unlocked ? g.Name : " " + g.UnlockHint + " ", 48,
        new Vector2(0, 50), TextAnchor.MiddleCenter, 520, 70).color = unlocked ? InkBlack : new Color(0.7f,
    MakeText(card.transform, unlocked ? g.Role : " ", 38,
        new Vector2(0, -50), TextAnchor.MiddleCenter, 520, 60).color = unlocked ? new Color(0.1f, 0.1f, 0.1f
    }
    MakeModeButtonCore(" ", new Vector2(0, -740), DoomGreen, ToggleCodex).transform.SetParent(_codexPanel, fals
    _codexPanel.gameObject.SetActive(false);
}
void ToggleCodex()
{
    bool show = !_codexPanel.gameObject.activeSelf;
    if (show)
    {
        //          /
        var generals = GeneralCodex.All;
        for (int i = 0; i < generals.Count; i++)
        {
            var card = _codexPanel.Find("Codex_" + generals[i].Id);
            if (card == null) continue;
            var img = card.GetComponent<Image>();
            bool unlocked = GeneralCodex.IsUnlocked(generals[i].Id);
            var g = generals[i];
            img.color = unlocked ? (g.Tier == GeneralCodex.Tier.Gold ? PaperYellow
                : g.Tier == GeneralCodex.Tier.Purple ? RevRed : DoomGreen) : LockedGray;
        }
    }
    _codexPanel.gameObject.SetActive(show);
}
void ShowToast(string message)
{
    _walletText.text = message;
    //          1.6s          .
    CancelInvoke(nameof(RestoreWallet));
    Invoke(nameof(RestoreWallet), 1.6f);
}
void RestoreWallet() => SyncWallet(force: true);
// —— UI          UGUI.  YAML ——
Button MakeModeButton(string text, Vector2 pos, Color color, System.Action onClick)
    => MakeModeButtonCore(text, pos, color, onClick, _canvas.transform);
Button MakeModeButtonCore(string text, Vector2 pos, Color color, System.Action onClick, Transform parent = null)
{
    var go = new GameObject("ModeBtn_" + text, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent != null ? parent : _canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(560, 130);
    var img = go.GetComponent<Image>();
    img.sprite = _square;

```

```
        img.color = color;
        var label = MakeText(go.transform, text, 44, Vector2.zero, TextAnchor.MiddleCenter, 560, 130);
        label.color = color == PaperYellow ? InkBlack : new Color(0.96f, 0.94f, 0.9f);
        var btn = go.GetComponent<Button>();
        btn.onClick.AddListener(() => onClick());
        return btn;
    }

    RectTransform MakeOverlayPanel()
    {
        var go = new GameObject("CodexPanel", typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        var img = go.GetComponent<Image>();
        img.sprite = _square;
        img.color = new Color(0.04f, 0.03f, 0.02f, 0.96f);
        return rt;
    }

    static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(w, h);
        var t = go.GetComponent<Text>();
        t.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
        t.text = text;
        t.fontSize = size;
        t.alignment = anchor;
        t.color = new Color(0.91f, 0.89f, 0.85f);
        t.supportRichText = true;
        return t;
    }

    static Sprite MakeSquareSprite()
    {
        var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
        var px = tex.GetPixels32();
        for (int i = 0; i < px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
        tex.SetPixels32(px);
        tex.Apply(false, false);
        return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
    }
}

// BannerWallet.cs
using System;
```

```

using System.Collections.Generic;
using System.Text;
using UnityEngine;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///         P02 LuckyWallet.T012-B
    ///
    ///
    ///         12 PlayerPrefs=
    ///         10+ x2 = x2+ ÷5
    /// </summary>
    public sealed class BannerWallet
    {
        public const string BalanceKey = "p03_banner_wallet";
        public const string RedeemedKey = "p03_generals";
        public const int ExchangeCost = 60;
        public int Balance { get; private set; }
        public List<int> RedeemedGeneralIds { get; } = new List<int>();
        BannerWallet() { }
        public static BannerWallet Load()
        {
            var w = new BannerWallet();
            w.Balance = PlayerPrefs.GetInt(BalanceKey, 0);
            string raw = PlayerPrefs.GetString(RedeemedKey, "");
            if (raw.Length > 0)
            {
                foreach (var part in raw.Split(','))
                {
                    if (int.TryParse(part, out int id)) w.RedeemedGeneralIds.Add(id);
                }
            }
            return w;
        }
        void Save()
        {
            PlayerPrefs.SetInt(BalanceKey, Balance);
            var sb = new StringBuilder();
            for (int i = 0; i < RedeemedGeneralIds.Count; i++)
            {
                if (i > 0) sb.Append(',');
                sb.Append(RedeemedGeneralIds[i]);
            }
            PlayerPrefs.SetString(RedeemedKey, sb.ToString());
            PlayerPrefs.Save();
        }
    }
    /// <summary>
    ///         =10+ x2+ ÷10 = x2+ ÷5
    /// </summary>
    public static int EarnFor(bool won, int waveReached, int kills)
    {
        int earned = waveReached * 2 + Math.Max(0, kills) / (won ? 10 : 5);
        if (won) earned += 10;
        return earned;
    }
}

```

```
public void Deposit(int coins)
{
    if (coins <= 0) return;
    Balance += coins;
    Save();
}

public bool CanExchange => Balance >= ExchangeCost;
/// <summary>    = -1    </summary>
public int TryExchange(int roll)
{
    if (!CanExchange) return -1;
    var pool = GeneralCodex.LockedIds();
    if (pool.Length == 0) return -1;
    int id = pool[(roll % pool.Length) + pool.Length % pool.Length];
    GeneralCodex.Unlock(id);
    RedeemedGeneralIds.Add(id);
    Balance = 0;
    Save();
    return id;
}

public static void ClearForTests()
{
    PlayerPrefs.DeleteKey(BalanceKey);
    PlayerPrefs.DeleteKey(RedeemedKey);
    GeneralCodex.ClearForTests();
}
}

}

// BattleLaunch.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P03.Newbie
{
    /// <summary>
    /// /
    /// S03_Battle BattleLauncher /
    /// = — S03_Battle
    /// </summary>
    public static class BattleLaunch
    {
        public enum Mode { Tutorial, Siege, Tianming }
        public static Mode Next = Mode.Tutorial;
        public static int NextSeed;
        /// <summary>    </summary>
        public static void StartRandomSiege()
        {
            Next = Mode.Siege;
            NextSeed = Mathf.Abs(RandomSeed()) % 100000 + 1;
        }
        /// <summary>    </summary>
    }
}
```

```

    public static void StartTianming()
    {
        Next = Mode.Tianming;
        NextSeed = TianmingPin.SeedForToday();
    }
    /// <summary>          false          </summary>
    public static bool StartFromFormationCode(string code)
    {
        var plan = FormationCodec.TryDecode(code);
        if (plan == null) return false;
        Next = Mode.Siege;
        NextSeed = plan.Value.Seed;
        return true;
    }
    public static void StartTutorial()
    {
        Next = Mode.Tutorial;
        NextSeed = 0;
    }
    /// <summary>Launcher          </summary>
    public static (Mode mode, int seed) Take()
    {
        var taken = (Next, NextSeed);
        Next = Mode.Tutorial;
        NextSeed = 0;
        return taken;
    }
    static int RandomSeed()
    {
        return UnityEngine.Random.Range(int.MinValue, int.MaxValue);
    }
}

// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P03.Newbie
{
    /// <summary>
    /// S03_Battle      BattleSim+TutorialFlow+BattleView  Tick
    ///   Sy.Telemetry Editor =Debug sink T001e      +TCB
    /// HUD              6
    /// V1.4      0-20261010-1245      BattleLaunch      —
    ///   .      /      SiegeLevel      /      +
    /// </summary>
    public sealed class BattleLauncher : MonoBehaviour
    {
        BattleSim _sim;
        TutorialFlow _tutorial;
        BattleView _view;
    }
}

```

```
Sy.Telemetry.TelemetryService _telemetry;
int _lastWave = -1, _lastLives = -1, _lastCrystals = -1, _lastKills = -1;
bool _tutorialMode = true;
void Awake()
{
    Application.targetFrameRate = 30;
    var canvas = FindObjectOfType<Canvas>();
    var (mode, seed) = BattleLaunch.Take();
    _tutorialMode = mode == BattleLaunch.Mode.Tutorial;
    if (_tutorialMode)
    {
        _sim = new BattleSim();
    }
    else
    {
        bool isTianming = mode == BattleLaunch.Mode.Tianming;
        _sim = new BattleSim(SiegeLevel.Generate(seed), isTianming);
    }
    _view = gameObject.AddComponent<BattleView>();
    _view.Initialize(canvas, _sim);
    _view.SetTotalWaves(_sim.TotalWaves);
    if (!_tutorialMode && _sim.Profile != null)
    {
        _view.SetCalamity("    " + _sim.Profile.CalamityName);
        _view.SetHint("    — " + _sim.Profile.CalamityName + " ");
    }
    _telemetry = new Sy.Telemetry.TelemetryService();
    _telemetry.AddSink(new TelemetryDebugSink());
    if (_tutorialMode)
    {
        _tutorial = new TutorialFlow(
            _sim,
            hint =>{
                Debug.Log("[HINT] " + hint);
                _view.SetHint(hint);
            },
            step =>{ _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
                new Dictionary<string, string>{ { "step", step.ToString() } });
            }
        );
    }
    else if (_sim.IsTianming)
    {
        _telemetry.Track("evt_tianming",
            TianmingGate.EntryFields("entry_click"));
    }
    _view.SlotTapped += OnSlotTapped;
    _view.TianjiPicked += OnTianjiPicked;
    _view.QingLongRequested += OnQingLongRequested;
    _view.EnemyTapped += OnEnemyTapped;
    _sim.TowerPlaced += OnTowerPlaced;
```

```
_sim.TianjiOffered += OnTianjiOffered;
_sim.QingLongFired += OnQingLongFired;
_sim.StateChanged += OnStateChanged;
}
void Update()
{
    _sim.Tick(Time.deltaTime);
    _view.SyncView(_sim);
    if (_sim.Wave != _lastWave) { _lastWave = _sim.Wave; _view.SetWave(_lastWave); }
    if (_sim.Lives != _lastLives) { _lastLives = _sim.Lives; _view.SetLives(_lastLives); }
    if (_sim.Crystals != _lastCrystals) { _lastCrystals = _sim.Crystals; _view.SetCrystals(_lastCrystals); }
    if (_sim.Kills != _lastKills) { _lastKills = _sim.Kills; _view.SetKills(_lastKills); }
}
void OnSlotTapped(int slot)
{
    _sim.PlaceTower(slot);
}
void OnTianjiPicked(int index)
{
    _sim.PickTianji(index);
}
void OnQingLongRequested()
{
    _sim.TryQingLongZhan();
}
void OnEnemyTapped(int enemyId)
{
    _sim.TapFocus(enemyId);
}
void OnTowerPlaced(BattleSim.TowerInst tower, int slot)
{
    _view.SetTower(slot, tower.IsGuanYu, tower.GeneralId);
}
void OnTianjiOffered()
{
    if (_tutorialMode) _view.ShowTianjiPanel();
    else _view.ShowTianjiPanel(_sim.TianjiEntries);
}
void OnQingLongFired()
{
    _view.PlaySlash();
}
void OnStateChanged(BattleSim.BattleState state)
{
    switch (state)
    {
        case BattleSim.BattleState.Prepare:
            _view.HideIntro();
            break;
        case BattleSim.BattleState.Won:

```

```

        Settle(true);
        break;
    case BattleSim.BattleState.Lost:
        Settle(false);
        break;
    }
}
void Settle(bool won)
{
    if (_tutorialMode)
    {
        _view.ShowSettle(won ? _sim.Wave : -1, _sim.Kills, _sim.Crystals);
        return;
    }
    var result = RunResult.Snapshot(_sim, won);
    BannerWallet.Load().Deposit(result.BannerEarned);
    TianmingBoard.TrySubmit(result, Sy.Core.ServiceLocator.TryGet(out Sy.Platform.IOpenDataBoard board) ? board
    _telemetry.Track("evt_tianming",
        TianmingGate.RunEndFields(result.Seed, result.Won, result.WaveReached));
    _view.ShowSettle(result);
}
sealed class TelemetryDebugSink : Sy.Telemetry.TelemetryService.ISink
{
    public void Write(string evtId, Dictionary<string, string> fields)
    {
        var sb = new System.Text.StringBuilder(evtId);
        foreach (var kv in fields)
        {
            sb.Append(' ').Append(kv.Key).Append('=').Append(kv.Value);
        }
        Debug.Log("[TELEMETRY] " + sb);
    }
}
}
}
// BattleSim.cs
using System;
using System.Collections.Generic;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///      C#      Play  Editor headless  NewbieFlowCheck
    ///      06 03  M0      →M1 L1      x2+  x1 → M2      +
    /// →M3      →M4      Boss+      →M5      +
    ///      A* S      5      .
    ///
    /// V1.4      CEO  0-20261010-1245      Profile != null
    /// SiegeLevel  —  /  /      3
    ///      +
    ///      =  +      Profile == null

```

```
/// NewbieFlowCheck
/// &lt;/summary&gt;
public sealed class BattleSim
{
    public enum BattleState { Intro, Prep, Fighting, TianjiPause, Won, Lost }
    public sealed class TowerInst
    {
        public int Id;
        public int Slot;
        public bool IsGuanYu;
        public int GeneralId;    // id 1 /2 /...    IsGuanYu
        public float Range;
        public float Atk;
        public float Interval;
        public float Cooldown;
    }
    public sealed class EnemyInst
    {
        public int Id;
        public bool Boss;    //
        public bool Elite;    //
        public int Seg;
        public float SegT;
        public float X, Y;
        public float Hp, MaxHp;
        public float Speed;
        public bool Dead;
    }
    public BattleState State { get; private set; } = BattleState.Intro;
    /// &lt;/summary&gt;    null=    &lt;/summary&gt;
    public SiegeLevel Profile { get; }
    public int Seed { get; }
    public bool IsTianming { get; private set; }
    public int TotalWaves =&gt; Profile != null ? Profile.TotalWaves : NewbieConfig.NewbieWaves;
    readonly List&lt;TowerInst&gt; _towers = new();
    readonly List&lt;EnemyInst&gt; _enemies = new();
    readonly float[] _seglens;
    int _nextId = 1;
    float _stateTimer;
    int _placed;
    int _waveSpawned;
    float _spawnTimer;
    bool _spawning;
    float _waveGapTimer;
    int _lastCleared = -1;
    bool _tianjiDone;
    int _tianjiCount;
    int[] _tianjiPool;    //
    int _focusId = -1;
    float _focusTimer;
```

```
float _enemySpeedMul = 1f;
float _atkMul = 1f;
float _towerIntervalMul = 1f;
float _crystalMul = 1f;
float _fireMul = 1f;          //
int _waveBonus;
public IReadOnlyList<TowerInst> Towers => _towers;
public IReadOnlyList<EnemyInst> Enemies => _enemies;
public int Wave { get; private set; }
public int Lives { get; private set; } = NewbieConfig.StartLives;
public int Crystals { get; private set; }
public int Kills { get; private set; }
public int PlacedCount => _placed;
public float BattleTime { get; private set; }
/// <summary>
//      = 0/1/2 = </summary>
public int[] TianjiEntries => _tianjiPool ?? new[] { 0, 1, 2 };
public event Action<TowerInst, int> TowerPlaced;
public event Action<int> WaveStarted;
public event Action<int> WaveCleared;
public event Action<EnemyInst> EnemySpawned;
public event Action<EnemyInst> EnemyKilled;
public event Action<int> LifeLost;
public event Action TianjiOffered;
public event Action<int> TianjiPicked;
public event Action BossKilled;
public event Action QingLongFired;
public event Action<BattleState> StateChanged;
public BattleSim() : this(null, false) { }
public BattleSim(SiegeLevel profile, bool isTianming)
{
    Profile = profile;
    IsTianming = isTianming;
    _segLens = new float[NewbieConfig.PathX.Length - 1];
    for (int s = 0; s < _segLens.Length; s++)
    {
        float dx = NewbieConfig.PathX[s + 1] - NewbieConfig.PathX[s];
        float dy = NewbieConfig.PathY[s + 1] - NewbieConfig.PathY[s];
        _segLens[s] = (float)Math.Sqrt(dx * dx + dy * dy);
    }
    if (profile != null)
    {
        Seed = profile.Seed;
        Lives = 3;
        Crystals = profile.StartCrystals;
        if (profile.CalamityId == 3) _crystalMul = 1f + profile.CalamityPower; //
        if (profile.CalamityId == 4) _towerIntervalMul = 1f + profile.CalamityPower; //
        var rng = new System.Random(profile.Seed ^ 0x5A11);
        var bag = new List<int> { 0, 1, 2, 3, 4, 5 };
        _tianjiPool = new int[3];
        for (int i = 0; i < 3; i++)
```

```

        {
            int at = rng.Next(bag.Count);
            _tianjiPool[i] = bag[at];
            bag.RemoveAt(at);
        }
    }
}

(int Count, float Hp, float Speed, bool Boss, bool Elite) WaveAt(int w)
{
    if (Profile == null)
    {
        var (tutCount, tutHp, tutSpeed) = NewbieConfig.GetWave(w);
        return (tutCount, tutHp, tutSpeed, w == NewbieConfig.NewbieWaves, false);
    }
    var def = Profile.GetWave(w);
    float hp = def.Hp * _fireMul;
    _fireMul = 1f;
    bool final = w == Profile.TotalWaves;
    bool elite = def.Boss && !final;
    return (def.Count, hp, def.Speed, final, elite);
}

/// <summary>      =      → →      = +      +      </summary>
public bool PlaceTower(int slot)
{
    bool prepOk = State == BattleState.Prepare
        || (Profile != null && State == BattleState.Fighting);
    if (!prepOk) return false;
    if (slot < 0 || slot >= NewbieConfig.SlotX.Length) return false;
    for (int i = 0; i < _towers.Count; i++)
        if (_towers[i].Slot == slot) return false;
    GeneralCodex.GeneralDef def;
    if (Profile == null)
    {
        if (_placed >= NewbieConfig.FreeTowerPlacements) return false;
        bool isGuanYu = _placed < NewbieConfig.FreeTowerPlacements - 1; //
        def = isGuanYu ? GeneralCodex.ById(1) : GeneralCodex.ById(2);
    }
    else
    {
        var unlocked = GeneralCodex.Unlocked();
        if (unlocked.Count == 0) return false;
        int cost = CostFor(unlocked[_placed % unlocked.Count].Tier);
        if (Crystals < cost) return false;
        def = unlocked[_placed % unlocked.Count];
        Crystals -= cost;
    }
    var tower = new TowerInst
    {
        Id = _nextId++,
        Slot = slot,

```

```

        IsGuanYu = def.Aoe,
        GeneralId = def.Id,
        Range = def.Range,
        Atk = def.Atk,
        Interval = def.Interval,
    };
    _towers.Add(tower);
    _placed++;
    TowerPlaced?.Invoke(tower, slot);
    return true;
}

static int CostFor(GeneralCodex.Tier tier) => tier switch
{
    GeneralCodex.Tier.Purple => 9,
    GeneralCodex.Tier.Gold => 12,
    _ => 6,
};

/// <summary> FocusSeconds </summary>
public void TapFocus(int enemyId)
{
    for (int i = 0; i < _enemies.Count; i++)
        if (_enemies[i].Id == enemyId)
        {
            _focusId = enemyId;
            _focusTimer = NewbieConfig.FocusSeconds;
            return;
        }
}

public bool IsFocused(EnemyInst e) => e.Id == _focusId && _focusTimer > 0f;
/// <summary> ≤40% → </summary>
public bool QingLongZhanReady
{
    get
    {
        for (int i = 0; i < _enemies.Count; i++)
        {
            var e = _enemies[i];
            if (e.Boss && e.Hp <= e.MaxHp * NewbieConfig.QingLongHpRatio) return true;
        }
        return false;
    }
}

/// <summary> M4 . </summary>
public bool TryQingLongZhan()
{
    if (State != BattleState.Fighting || !QingLongZhanReady) return false;
    for (int i = 0; i < _enemies.Count; i++)
    {
        var e = _enemies[i];
        if (!e.Boss) continue;
    }
}

```

```
        QingLongFired?.Invoke();
        DamageEnemy(e, e.Hp + 1f);
        return true;
    }
    return false;
}

public void PickTianji(int index)
{
    if (State != BattleState.TianjiPause) return;
    int entry;
    if (Profile == null)
    {
        entry = index;    //    0/1/2
    }
    else
    {
        if (index < 0 || index >= _tianjiPool.Length) return;
        entry = _tianjiPool[index];
    }
    switch (entry)
    {
        case 0: _enemySpeedMul = 0.8f; break;           //
        case 1: _atkMul = 1.25f; break;                 //
        case 2: _waveBonus = 10; break;                 //
        case 3: Crystals += (int)(15 * _crystalMul); break; //
        case 4: Lives += 1; break;                     //
        case 5: _fireMul = 0.85f; break;                 //
    }
    _waveGapTimer = 0f;
    TianjiPicked?.Invoke(entry);
    SetState(BattleState.Fighting);
}

public void Tick(float dt)
{
    switch (State)
    {
        case BattleState.Intro:
            _stateTimer += dt;
            if (_stateTimer >= NewbieConfig.IntroSeconds) SetState(BattleState.Prep);
            return;
        case BattleState.Prep:
            _stateTimer += dt;
            int need = Profile != null ? 1 : NewbieConfig.FreeTowerPlacements;
            if (_placed >= 1 && (_placed >= need || _stateTimer >= NewbieConfig.PrepStartSecond)
            {
                SetState(BattleState.Fighting);
                BeginWave(1);
            }
            return;
        case BattleState.Fighting:
```

```
BattleTime += dt;
if (_focusTimer > 0f) _focusTimer -= dt;
TickSpawns(dt);
TickEnemies(dt);
TickTowers(dt);
TickWaves(dt);
if (Lives <= 0) SetState(BattleState.Lost);
return;
}
}
void BeginWave(int w)
{
    Wave = w;
    var (_, _, boss, elite) = WaveAt(w);
    _waveSpawned = 0;
    _spawnTimer = 0f;
    _waveGapTimer = 0f;
    _spawning = true;
    WaveStarted?.Invoke(w);
    float bossHp = Profile != null
        ? NewbieConfig.BossHp * (1f + 0.12f * Profile.TotalWaves)
        : NewbieConfig.BossHp;
    if (boss || elite)
    {
        var e = new EnemyInst
        {
            Id = _nextId++,
            Boss = boss,
            Elite = elite,
            Seg = 0,
            SegT = 0f,
            X = NewbieConfig.PathX[0],
            Y = NewbieConfig.PathY[0],
            Hp = boss ? bossHp : bossHp * 0.55f,
            MaxHp = boss ? bossHp : bossHp * 0.55f,
            Speed = elite ? NewbieConfig.BossSpeed * 1.25f : NewbieConfig.BossSpeed,
        };
        _enemies.Add(e);
        EnemySpawned?.Invoke(e);
    }
}
void TickSpawns(float dt)
{
    if (!_spawning) return;
    var (count, hp, speed, _, _) = WaveAt(Wave);
    float interval = Profile != null ? Profile.SpawnInt : NewbieConfig.SpawnInterval;
    _spawnTimer += dt;
    while (_spawnTimer >= interval && _waveSpawned < count)
    {
        _spawnTimer -= interval;
    }
}
```

```
_waveSpawned++;
var e = new EnemyInst
{
    Id = _nextId++,
    Seg = 0,
    SegT = 0f,
    X = NewbieConfig.PathX[0],
    Y = NewbieConfig.PathY[0],
    Hp = hp,
    MaxHp = hp,
    Speed = speed,
};
_enemies.Add(e);
EnemySpawned?.Invoke(e);
}
if (_waveSpawned >= count) _spawning = false;
}
void TickEnemies(float dt)
{
    for (int i = _enemies.Count - 1; i >= 0; i--)
    {
        var e = _enemies[i];
        float move = e.Speed * _enemySpeedMul * dt;
        while (move > 0f)
        {
            float segLen = _segLens[e.Seg];
            float remain = (1f - e.SegT) * segLen;
            if (move < remain)
            {
                e.SegT += move / segLen;
                move = 0f;
            }
            else
            {
                move -= remain;
                e.Seg++;
                e.SegT = 0f;
                if (e.Seg >= _segLens.Length)
                {
                    // =
                    if (e.Boss)
                    {
                        e.Seg = 0;
                        e.SegT = 0f;
                        Lives = Math.Max(0, Lives - 1);
                        LifeLost?.Invoke(Lives);
                    }
                    else
                    {
                        _enemies.RemoveAt(i);
                    }
                }
            }
        }
    }
}
```

```
Lives = Math.Max(0, Lives - 1);
LifeLost?.Invoke(Lives);
    }
    break;
}
}
}
if (e.Seg < _segLens.Length)
{
    e.X = Lerp(NewbieConfig.PathX[e.Seg], NewbieConfig.PathX[e.Seg + 1], e.SegT);
    e.Y = Lerp(NewbieConfig.PathY[e.Seg], NewbieConfig.PathY[e.Seg + 1], e.SegT);
}
}
}
static float Lerp(float a, float b, float t) => a + (b - a) * t;
void TickTowers(float dt)
{
    for (int i = 0; i < _towers.Count; i++)
    {
        var t = _towers[i];
        t.Cooldown -= dt;
        if (t.Cooldown > 0f) continue;
        bool any = false;
        if (t.IsGuanYu)
        {
            //
            float r2 = t.Range * t.Range;
            for (int j = _enemies.Count - 1; j >= 0; j--)
            {
                var e = _enemies[j];
                float dx = e.X - NewbieConfig.SlotX[t.Slot], dy = e.Y - NewbieConfig.SlotY[t.Slot];
                if (dx * dx + dy * dy > r2) continue;
                any = true;
                DamageEnemy(e, t.Atk * _atkMul);
            }
        }
        else
        {
            //
            float r2 = t.Range * t.Range;
            EnemyInst target = null;
            float best = float.MaxValue;
            for (int j = 0; j < _enemies.Count; j++)
            {
                var e = _enemies[j];
                float dx = e.X - NewbieConfig.SlotX[t.Slot], dy = e.Y - NewbieConfig.SlotY[t.Slot];
                float d2 = dx * dx + dy * dy;
                if (d2 > r2) continue;
                if (IsFocused(e)) { target = e; best = -1f; break; }
                if (d2 < best) { best = d2; target = e; }
            }
        }
    }
}
```

```
        }
        if (target != null)
        {
            any = true;
            DamageEnemy(target, t.Atk * _atkMul);
        }
    }
    t.Cooldown = any ? t.Interval * _towerIntervalMul : 0.05f;
}

void DamageEnemy(EnemyInst e, float dmg)
{
    e.Hp -= dmg;
    if (e.Hp > 0f) return;
    e.Dead = true;
    _enemies.Remove(e);
    Kills++;
    Crystals += (int)(NewbieConfig.KillCrystal * _crystalMul);
    if (e.Boss)
    {
        //
        for (int i = _enemies.Count - 1; i >= 0; i--)
        {
            var rest = _enemies[i];
            rest.Dead = true;
            _enemies.RemoveAt(i);
            Kills++;
            Crystals += (int)(NewbieConfig.KillCrystal * _crystalMul);
            EnemyKilled?.Invoke(rest);
        }
        BossKilled?.Invoke();
        EnemyKilled?.Invoke(e);
        SetState(BattleState.Won);
        return;
    }
    EnemyKilled?.Invoke(e);
}

void TickWaves(float dt)
{
    if (_spawning || _enemies.Count > 0) return;
    _waveGapTimer += dt;
    float gap = Profile != null ? Profile.WaveGap : NewbieConfig.WaveGapSeconds;
    if (_waveGapTimer < gap) return;
    int cleared = Wave;
    if (cleared >= TotalWaves) return; // Boss
    if (_lastCleared != cleared)
    {
        _lastCleared = cleared;
        int bonus = NewbieConfig.WaveClearCrystal + _waveBonus;
        if (Profile == null) Crystals += bonus; //
```

```

        else Crystals += (int)(bonus * _crystalMul); //
        WaveCleared?.Invoke(cleared);
    }
    bool offer;
    if (Profile == null)
        offer = cleared == 2 && !_tianjiDone;
    else
        offer = _tianjiCount < _tianjiPool.Length;
    if (offer)
    {
        if (Profile == null) _tianjiDone = true;
        else _tianjiCount++;
        SetState(BattleState.TianjiPause);
        TianjiOffered?.Invoke();
        return;
    }
    BeginWave(cleared + 1);
}
void SetState(BattleState s)
{
    if (State == s) return;
    State = s;
    _stateTimer = 0f;
    StateChanged?.Invoke(s);
}
}
}
// BattleView.cs
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///     UGUI— x x x
    ///     UI Initialize      YAML      =      /      /      /
    ///
    /// </summary>
    public sealed class BattleView : MonoBehaviour
    {
        Canvas _canvas;
        Sprite _square;
        readonly Image[] _slotImages = new Image[NewbieConfig.SlotX.Length];
        readonly Text[] _slotLabels = new Text[NewbieConfig.SlotX.Length];
        Text _waveText, _livesText, _crystalText, _killsText, _hintText, _calamityText;
        Button _qingLongButton;
        Image _qingLongImage;
        RectTransform _introOverlay, _tianjiPanel, _slashOverlay, _settlePanel;
    }
}

```

```
Text _settleStats;
Text _settleTitle;
int _totalWaves = NewbieConfig.NewbieWaves;
readonly Text[] _tianjiNames = new Text[3];
readonly Text[] _tianjiDescs = new Text[3];
readonly Dictionary<int, EnemyVisual> _enemies = new();
readonly List<int> _scratch = new();
public event Action<int> SlotTapped;
public event Action<int> EnemyTapped;
public event Action<int> TianjiPicked;
public event Action QingLongRequested;
static readonly Color RevRed = new Color(0.757f, 0.153f, 0.176f); // #C1272D
static readonly Color PaperYellow = new Color(0.929f, 0.871f, 0.706f); //
static readonly Color InkBlack = new Color(0.043f, 0.043f, 0.047f); //
static readonly Color DoomGreen = new Color(0.298f, 0.686f, 0.314f); // #4CAF50
static readonly Color PathCol = new Color(0.16f, 0.08f, 0.08f);
public void Initialize(Canvas canvas, BattleSim sim)
{
    _canvas = canvas;
    _square = MakeSquareSprite();
    BuildBackdrop();
    BuildHud();
    BuildPath();
    BuildSlots();
    BuildIntroOverlay();
    BuildTianjiPanel();
    BuildSlashOverlay();
    BuildSettlePanel();
    SetHint(" .....");
}
// -----
/// <summary>X027 generated art: fullscreen backdrop behind all UI; skipped
/// entirely when the art key is missing so headless checks stay green.</summary>
void BuildBackdrop()
{
    var art = Sy.Core.ArtRuntime.Get("P03_scene_01");
    if (art == null) return;
    var go = new GameObject("ArtBackdrop", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    rt.SetAsFirstSibling();
    var image = go.GetComponent<Image>();
    image.sprite = art;
    image.color = Color.white;
    image.preserveAspect = true;
    image.raycastTarget = false;
}
```

```

void BuildHud()
{
    var root = _canvas.transform;
    _waveText = MakeText(root, "L1    0/4", 46, new Vector2(0, 880), TextAnchor.UpperLeft, 380, 70);
    _waveText.color = PaperYellow;
    _livesText = MakeText(root, "    5", 46, new Vector2(0, 880), TextAnchor.UpperRight, 340, 70);
    _crystalText = MakeText(root, "    0", 44, new Vector2(0, 820), TextAnchor.UpperLeft, 380, 60);
    _crystalText.color = PaperYellow;
    _killsText = MakeText(root, "    0", 44, new Vector2(0, 820), TextAnchor.UpperRight, 380, 60);
    _calamityText = MakeText(root, "", 36, new Vector2(0, 760), TextAnchor.UpperLeft, 520, 50);
    _calamityText.color = DoomGreen;
    _hintText = MakeText(root, "", 40, new Vector2(0, -780), TextAnchor.MiddleCenter, 1020, 150);
    _hintText.color = new Color(0.95f, 0.87f, 0.6f);
    var btnGo = new GameObject("QingLongButton", typeof(Image), typeof(Button));
    btnGo.transform.SetParent(root, false);
    var btnRt = (RectTransform)btnGo.transform;
    btnRt.anchorMin = btnRt.anchorMax = new Vector2(0.5f, 0f);
    btnRt.anchoredPosition = new Vector2(0, 120);
    btnRt.sizeDelta = new Vector2(460, 140);
    _qingLongImage = btnGo.GetComponent<Image>();
    _qingLongImage.sprite = _square;
    _qingLongImage.color = new Color(0.35f, 0.35f, 0.35f);
    _qingLongButton = btnGo.GetComponent<Button>();
    _qingLongButton.interactable = false;
    MakeText(btnGo.transform, "    ", 54, Vector2.zero, TextAnchor.MiddleCenter, 460, 140).color = InkBlack;
    _qingLongButton.onClick.AddListener(() => QingLongRequested?.Invoke());
}

void BuildPath()
{
    for (int s = 0; s < NewbieConfig.PathX.Length - 1; s++)
    {
        float x0 = NewbieConfig.PathX[s], y0 = NewbieConfig.PathY[s];
        float x1 = NewbieConfig.PathX[s + 1], y1 = NewbieConfig.PathY[s + 1];
        bool horiz = Math.Abs(y1 - y0) < 1f;
        var go = new GameObject("PathSeg_" + s, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2((x0 + x1) * 0.5f, (y0 + y1) * 0.5f);
        rt.sizeDelta = horiz
            ? new Vector2(Math.Abs(x1 - x0) + 90f, 90f)
            : new Vector2(90f, Math.Abs(y1 - y0) + 90f);
        var img = go.GetComponent<Image>();
        img.sprite = _square;
        img.color = PathCol;
    }
    MakeText(_canvas.transform, "    ", 34, new Vector2(-460f, 820f), TextAnchor.MiddleCenter, 260, 50).color =
    MakeText(_canvas.transform, "    ", 34, new Vector2(-500f, -660f), TextAnchor.MiddleCenter, 260, 50).color = P
}

void BuildSlots()

```

```

{
    for (int i = 0; i < NewbieConfig.SlotX.Length; i++)
    {
        int index = i;
        var go = new GameObject("Slot_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(_canvas.transform, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2(NewbieConfig.SlotX[i], NewbieConfig.SlotY[i]);
        rt.sizeDelta = new Vector2(140, 140);
        var img = go.GetComponent<Image>();
        var slotArt = Sy.Core.ArtRuntime.Get("P03_unit_01");
        if (slotArt != null)
        {
            img.sprite = slotArt;
            img.color = Color.white;
            img.preserveAspect = true;
        }
        else
        {
            img.sprite = _square;
            img.color = new Color(0.13f, 0.13f, 0.13f, 0.9f);
        }
        _slotImages[i] = img;
        _slotLabels[i] = MakeText(go.transform, " ", 34, Vector2.zero, TextAnchor.MiddleCenter, 140, 140);
        _slotLabels[i].color = new Color(0.55f, 0.52f, 0.45f);
        go.GetComponent<Button>().onClick.AddListener(() => SlotTapped?.Invoke(index));
    }
}

void BuildIntroOverlay()
{
    _introOverlay = MakeOverlayPanel("IntroOverlay", new Color(0.06f, 0.02f, 0.02f, 0.95f));
    MakeText(_introOverlay, " ", 100, new Vector2(0, 100), TextAnchor.MiddleCenter, 1000, 130).color = Rev
    MakeText(_introOverlay, " · L1 ", 46, new Vector2(0, -60), TextAnchor.MiddleCenter, 1000, 80).color =
}

void BuildTianjiPanel()
{
    _tianjiPanel = MakeOverlayPanel("TianjiPanel", new Color(0.03f, 0.03f, 0.03f, 0.92f));
    MakeText(_tianjiPanel, " · ", 76, new Vector2(0, 560), TextAnchor.MiddleCenter, 1000, 100).color = Pape
    for (int i = 0; i < 3; i++)
    {
        int index = i;
        var card = NewbieConfig.TianjiCards[i];
        var go = new GameObject("Tianji_" + i, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(_tianjiPanel, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = new Vector2(-330 + i * 330, 0);
        rt.sizeDelta = new Vector2(300, 420);
        var img = go.GetComponent<Image>();

```

```

        img.sprite = _square;
        img.color = i == 0 ? new Color(0.55f, 0.12f, 0.12f) : new Color(0.14f, 0.16f, 0.14f);    //
        _tianjiNames[i] = MakeText(go.transform, " " + card.Name + " ", 56, new Vector2(0, 120), TextAnchor.Midd
        _tianjiNames[i].color = PaperYellow;
        _tianjiDescs[i] = MakeText(go.transform, card.Desc, 38, new Vector2(0, -40), TextAnchor.MiddleCenter, 28
        _tianjiDescs[i].color = new Color(0.85f, 0.85f, 0.8f);
        go.GetComponent<Button>().onClick.AddListener(() =>{
            _tianjiPanel.gameObject.SetActive(false);
            TianjiPicked?.Invoke(index);
        });
    }
    MakeText(_tianjiPanel, "      —      ", 40, new Vector2(0, -400), TextAnchor.MiddleCenter, 1000, 80).color
    _tianjiPanel.gameObject.SetActive(false);
}
/// <summary>          0=          </summary>
public void ShowTianjiPanel(int[] entries)
{
    for (int i = 0; i < 3 && i < entries.Length; i++)
    {
        var card = NewbieConfig.TianjiCards[Math.Max(0, Math.Min(NewbieConfig.TianjiCards.Length - 1, entries[i]
        _tianjiNames[i].text = " " + card.Name + " ";
        _tianjiDescs[i].text = card.Desc;
    }
    ShowTianjiPanel();
}
void BuildSlashOverlay()
{
    _slashOverlay = MakeOverlayPanel("SlashOverlay", new Color(1f, 0.98f, 0.92f, 0.75f));
    var lineGo = new GameObject("SlashLine", typeof(RectTransform), typeof(Image));
    lineGo.transform.SetParent(_slashOverlay, false);
    var lineRt = (RectTransform)lineGo.transform;
    lineRt.anchorMin = lineRt.anchorMax = new Vector2(0.5f, 0.5f);
    lineRt.anchoredPosition = Vector2.zero;
    lineRt.sizeDelta = new Vector2(2200, 16);
    lineRt.localRotation = Quaternion.Euler(0f, 0f, 24f);
    var lineImg = lineGo.GetComponent<Image>();
    lineImg.sprite = _square;
    lineImg.color = RevRed;
    MakeText(_slashOverlay, "      ", 110, new Vector2(0, -180), TextAnchor.MiddleCenter, 1000, 140).color = Ink
    _slashOverlay.gameObject.SetActive(false);
}
void BuildSettlePanel()
{
    _settlePanel = MakeOverlayPanel("SettlePanel", new Color(0.03f, 0.03f, 0.03f, 0.96f));
    _settleTitle = MakeText(_settlePanel, "L1      ", 90, new Vector2(0, 560), TextAnchor.MiddleCenter, 1040, 110
    _settleTitle.color = PaperYellow;
    _settleStats = MakeText(_settlePanel, "", 46, new Vector2(0, 380), TextAnchor.MiddleCenter, 1000, 260);
    MakeActionButton(_settlePanel, "      ", new Vector2(-260, -260), () => UnityEngine.SceneManagement.SceneMa
    MakeActionButton(_settlePanel, "      ", new Vector2(260, -260), () => UnityEngine.SceneManagement.SceneMa

```

```
        _settlePanel.gameObject.SetActive(false);
    }
    // ----- reconcile -----
    public void SyncView(BattleSim sim)
    {
        ReconcileEnemies(sim);
        foreach (var kv in _enemies)
        {
            var v = kv.Value;
            v.Rt.anchoredPosition = new Vector2(v.X, v.Y);
            if (v.Fill != null) v.Fill.fillAmount = v.MaxHp > 0f ? v.Hp / v.MaxHp : 0f;
            bool focused = v.Focused;
            if (focused != v.LastFocused)
            {
                v.LastFocused = focused;
                v.FocusLabel.gameObject.SetActive(focused);
            }
        }
        bool ready = sim.QingLongZhanReady;
        if (_qingLongButton.interactable != ready)
        {
            _qingLongButton.interactable = ready;
            _qingLongImage.color = ready ? RevRed : new Color(0.35f, 0.35f, 0.35f);
        }
    }
    void ReconcileEnemies(BattleSim sim)
    {
        _scratch.Clear();
        foreach (var kv in _enemies)
            if (!ContainsEnemy(sim, kv.Key)) _scratch.Add(kv.Key);
        for (int i = 0; i < _scratch.Count; i++)
        {
            Destroy(_enemies[_scratch[i]].Go);
            _enemies.Remove(_scratch[i]);
        }
        for (int i = 0; i < sim.Enemies.Count; i++)
        {
            var e = sim.Enemies[i];
            if (!_enemies.TryGetValue(e.Id, out var v))
            {
                v = CreateEnemyVisual(e);
                _enemies[e.Id] = v;
            }
            v.X = e.X;
            v.Y = e.Y;
            v.Hp = e.Hp;
            v.MaxHp = e.MaxHp;
            v.Focused = sim.IsFocused(e);
        }
    }
}
```

```

static bool ContainsEnemy(BattleSim sim, int id)
{
    for (int i = 0; i < sim.Enemies.Count; i++)
        if (sim.Enemies[i].Id == id) return true;
    return false;
}

EnemyVisual CreateEnemyVisual(BattleSim EnemyInst e)
{
    int id = e.Id;
    var go = new GameObject("Enemy_" + e.Id, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.sizeDelta = e.Boss ? new Vector2(160, 160) : (e.Elite ? new Vector2(110, 110) : new Vector2(70, 70));
    var image = go.GetComponent<Image>();
    image.sprite = _square;
    image.color = e.Boss ? PaperYellow : (e.Elite ? new Color(0.78f, 0.55f, 0.25f) : new Color(0.35f, 0.42f, 0.35f));
    if (e.Boss)
        MakeText(go.transform, " ", 64, Vector2.zero, TextAnchor.MiddleCenter, 160, 160).color = InkBlack;
    else if (e.Elite)
        MakeText(go.transform, " ", 44, Vector2.zero, TextAnchor.MiddleCenter, 110, 110).color = InkBlack;
    Image fill = null;
    var hpGo = new GameObject("Hp", typeof(RectTransform), typeof(Image));
    hpGo.transform.SetParent(go.transform, false);
    var hpRt = (RectTransform)hpGo.transform;
    hpRt.anchorMin = new Vector2(0f, 1f);
    hpRt.anchorMax = new Vector2(1f, 1f);
    hpRt.anchoredPosition = new Vector2(0, -6);
    hpRt.sizeDelta = new Vector2(-8, 10);
    fill = hpGo.GetComponent<Image>();
    fill.sprite = _square;
    fill.type = Image.Type.Filled;
    fill.fillMethod = Image.FillMethod.Horizontal;
    fill.color = new Color(0.2f, 0.9f, 0.3f);
    var focusLabel = MakeText(go.transform, " ", 34, new Vector2(0, 120), TextAnchor.MiddleCenter, 220, 50);
    focusLabel.color = PaperYellow;
    focusLabel.gameObject.SetActive(false);
    go.GetComponent<Button>().onClick.AddListener(() => EnemyTapped?.Invoke(id));
    return new EnemyVisual
    {
        Go = go,
        Rt = rt,
        Fill = fill,
        FocusLabel = focusLabel.gameObject,
        X = e.X,
        Y = e.Y,
        Hp = e.Hp,
        MaxHp = e.MaxHp,
    };
}

```

```

// —— HUD ——

public void SetWave(int w) => _waveText.text = " " + w + "/" + _totalWaves;
public void SetTotalWaves(int total) { _totalWaves = total & 0 ? total : NewbieConfig.NewbieWaves; }
public void SetCalamity(string name) => _calamityText.text = name;
public void SetLives(int l) => _livesText.text = " " + l;
public void SetCrystals(int c) => _crystalText.text = " " + c;
public void SetKills(int k) => _killsText.text = " " + k;
public void SetHint(string hint) => _hintText.text = hint;
public void HideIntro() => _introOverlay.gameObject.SetActive(false);
public void ShowTianjiPanel() => _tianjiPanel.gameObject.SetActive(true);
public void SetTower(int slot, bool isGuanYu, int generalId)
{
    _slotImages[slot].color = isGuanYu ? RevRed : DoomGreen;
    var def = GeneralCodexById(generalId);
    string label = def != null & def.Name.Length & 0 ? def.Name.Substring(0, 1) : (isGuanYu ? " " :
    _slotLabels[slot].text = label;
    _slotLabels[slot].color = InkBlack;
    _slotLabels[slot].fontSize = 64;
}
public void PlaySlash()
{
    StartCoroutine(SlashRoutine());
}
IEnumerator SlashRoutine()
{
    _slashOverlay.gameObject.SetActive(true);
    yield return new WaitForSeconds(0.5f); // 06 M4
    _slashOverlay.gameObject.SetActive(false);
}
public void ShowSettle(int wave, int kills, int crystals)
{
    _settlePanel.gameObject.SetActive(true);
    _settleStats.text = wave & 0
        ? $" {wave}/{NewbieConfig.NewbieWaves} · {kills} · {crystals}\n<size=40> L1-DEMO-0001
        : $" ..... {kills} · {crystals}\n<size=40> — </size>";
}
/// <summary> / + </summary>
public void ShowSettle(RunResult result)
{
    _settleTitle.text = result.Won ? " " : " ";
    _settleStats.text = result.Won
        ? $" {result.WaveReached}/{_totalWaves} · {result.Kills} · +{result.BannerEarned}\n<size=40>
        : $" {result.WaveReached}/{_totalWaves} · {result.Kills} · +{result.BannerEarned}\n<size=40>
    _settlePanel.gameObject.SetActive(true);
}
// —— ——
RectTransform MakeOverlayPanel(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
}

```

```
        var rt = (RectTransform)go.transform;
        rt.anchorMin = Vector2.zero;
        rt.anchorMax = Vector2.one;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        var image = go.GetComponent<Image>();
        image.sprite = _square;
        image.color = color;
        return rt;
    }

    void MakeActionButton(RectTransform parent, string text, Vector2 pos, Action onClick)
    {
        var go = new GameObject(text, typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(420, 120);
        var image = go.GetComponent<Image>();
        image.sprite = _square;
        image.color = DoomGreen;
        MakeText(go.transform, text, 46, Vector2.zero, TextAnchor.MiddleCenter, 420, 120);
        go.GetComponent<Button>().onClick.AddListener(() => onClick());
    }

    static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(w, h);
        var t = go.GetComponent<Text>();
        t.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
        t.text = text;
        t.fontSize = size;
        t.alignment = anchor;
        t.color = new Color(0.91f, 0.89f, 0.85f);
        t.supportRichText = true;
        return t;
    }

    static Sprite MakeSquareSprite()
    {
        var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
        var px = tex.GetPixels32();
        for (int i = 0; i < px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
        tex.SetPixels32(px);
        tex.Apply(false, false);
        return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
    }

    // _____
```

```

sealed class EnemyVisual
{
    public GameObject Go;
    public RectTransform Rt;
    public Image Fill;
    public GameObject FocusLabel;
    public bool LastFocused;
    public bool Focused;
    public float X, Y;
    public float Hp, MaxHp;
}
}

// FormationCodec.cs
using System;
using System.Collections.Generic;
using System.Text;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///     P02 DefenseCodec .
    ///     base36+
    ///     06 03 ① = C#
    ///     3bit+ 1bit=0..15 &lt; 36
    /// </summary>
    public static class FormationCodec
    {
        const string Alphabet = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        const int Version = 1;
        /// <summary> V(1)+ (6)+ (1)+ 1 + (1) </summary>
        public static string Encode(int seed, IReadOnlyList<BattleSim.TowerInst> towers)
        {
            var sb = new StringBuilder();
            sb.Append(Digit(Version));
            AppendUnsigned(sb, seed > 0 ? (uint)seed : 0u, 6);
            sb.Append(Digit(towers.Count));
            for (int i = 0; i < towers.Count; i++)
                sb.Append(Digit(((towers[i].Slot & 0x7) < 1) | (towers[i].IsGuanYu ? 1 : 0)));
            sb.Append(ChecksumChar(sb.ToString()));
            return sb.ToString();
        }

        public static string Encode(BattleSim sim)
            => Encode(sim.Seed, sim.Towers);
        /// <summary> null </summary>
        public static FormationPlan? TryDecode(string code)
        {
            if (string.IsNullOrEmpty(code) || code.Length < 9) return null;
            string body = code.Substring(0, code.Length - 1);
            if (code[code.Length - 1] != ChecksumChar(body)) return null;
            int at = 0;

```

```
int version = Value(body[at++]);
if (version != Version) return null;
uint seed = ReadUnsigned(body, ref at, 6);
int count = Value(body[at++]);
if (count < 0 || count > 8) return null;
if (body.Length - at != count) return null;
var slots = new int[count];
bool[] guan = new bool[count];
for (int i = 0; i < count; i++)
{
    int cell = Value(body[at++]);
    if (cell < 0) return null;
    slots[i] = (cell >> 1) & 0x7;
    guan[i] = (cell & 1) == 1;
}
return new FormationPlan((int)seed, slots, guan);
}
public readonly struct FormationPlan
{
    public readonly int Seed;
    public readonly int[] Slots;
    public readonly bool[] IsGuanYu;
    public FormationPlan(int seed, int[] slots, bool[] isGuanYu)
    {
        Seed = seed;
        Slots = slots;
        IsGuanYu = isGuanYu;
    }
}
static char Digit(int v) => Alphabet[v % 36];
static int Value(char c) => Alphabet.IndexOf(char.ToUpperInvariant(c));
static void AppendUnsigned(StringBuilder sb, uint v, int chars)
{
    char[] buf = new char[chars];
    uint x = v;
    for (int i = chars - 1; i >= 0; i--)
    {
        buf[i] = Alphabet[(int)(x % 36u)];
        x /= 36u;
    }
    for (int i = 0; i < chars; i++) sb.Append(buf[i]);
}
static uint ReadUnsigned(string s, ref int at, int chars)
{
    uint v = 0;
    for (int i = 0; i < chars && at < s.Length; i++)
        v = v * 36u + (uint)Math.Max(0, Value(s[at++]));
    return v;
}
static char ChecksumChar(string body)
```

```

        {
            int sum = 0;
            for (int i = 0; i < body.Length; i++) sum += body[i];
            return Alphabet[sum % 36];
        }
    }
}

// GeneralCodex.cs
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///     V1.4      8    — /
    ///     BannerWallet      =
    ///     12      PlayerPrefs
    /// </summary>
    public static class GeneralCodex
    {
        public enum Tier { Common, Purple, Gold }
        public sealed class GeneralDef
        {
            public int Id;
            public string Name;      //      +
            public string Role;      //      .
            public Tier Tier;
            public float Range;
            public float Atk;
            public float Interval;
            public bool AoE;
            public string UnlockHint;
            public GeneralDef(int id, string name, string role, Tier tier,
                float range, float atk, float interval, bool aoe, string unlockHint)
            {
                Id = id; Name = name; Role = role; Tier = tier;
                Range = range; Atk = atk; Interval = interval; AoE = aoe;
                UnlockHint = unlockHint;
            }
        }
        static readonly GeneralDef[] Generals =
        {
            new GeneralDef(1, " ", " ", Tier.Common, 280f, 12f, 0.8f, true, " "),
            new GeneralDef(2, " ", " ", Tier.Common, 520f, 26f, 1.1f, false, " "),
            new GeneralDef(3, " ", " ", Tier.Purple, 360f, 34f, 1.0f, false, " "),
            new GeneralDef(4, " ", " ", Tier.Purple, 480f, 10f, 1.2f, true, " "),
            new GeneralDef(5, " ", " ", Tier.Gold, 300f, 22f, 0.9f, true, " "),
            new GeneralDef(6, " ", " ", Tier.Gold, 380f, 18f, 0.45f, false, " "),
            new GeneralDef(7, " ", " ", Tier.Purple, 320f, 40f, 1.4f, false, " "),
            new GeneralDef(8, " ", " ", Tier.Gold, 460f, 16f, 1.0f, false, " "),
        }
    }
}

```

```
};
const string UnlockedKey = "p03_codex_unlocked";
public static IReadOnlyList<GeneralDef> All => Generals;
public static GeneralDef ById(int id)
{
    for (int i = 0; i < Generals.Length; i++)
        if (Generals[i].Id == id) return Generals[i];
    return null;
}
static HashSet<int> LoadUnlocked()
{
    var set = new HashSet<int>();
    string raw = PlayerPrefs.GetString(UnlockedKey, "");
    if (raw.Length == 0)
    {
        set.Add(1);
        set.Add(2);
        return set;
    }
    foreach (var part in raw.Split(','))
        if (int.TryParse(part, out int id)) set.Add(id);
    return set;
}
static void SaveUnlocked(HashSet<int> set)
{
    var sb = new System.Text.StringBuilder();
    int n = 0;
    foreach (int id in set)
    {
        if (n++ > 0) sb.Append(',');
        sb.Append(id);
    }
    PlayerPrefs.SetString(UnlockedKey, sb.ToString());
    PlayerPrefs.Save();
}
public static bool IsUnlocked(int id)
    => LoadUnlocked().Contains(id);
/// <summary>false</summary>
public static bool Unlock(int id)
{
    var set = LoadUnlocked();
    if (!set.Add(id)) return false;
    SaveUnlocked(set);
    return true;
}
/// <summary></summary>
public static List<GeneralDef> Unlocked()
{
    var set = LoadUnlocked();
    var list = new List<GeneralDef>();
```

```

        for (int i = 0; i < Generals.Length; i++)
            if (set.Contains(Generals[i].Id)) list.Add(Generals[i]);
        return list;
    }
    /// <summary>
    /// = </summary>
    public static int[] LockedIds()
    {
        var set = LoadUnlocked();
        var ids = new List<int>();
        for (int i = 0; i < Generals.Length; i++)
            if (!set.Contains(Generals[i].Id)) ids.Add(Generals[i].Id);
        return ids.ToArray();
    }
    public static void ClearForTests() => PlayerPrefs.DeleteKey(UnlockedKey);
}
}
// NewbieConfig.cs
namespace Sy.P03.Newbie
{
    public readonly struct TowerDef
    {
        public readonly string Name;
        public readonly float Range;
        public readonly float Atk;
        public readonly float Interval;
        public readonly bool Aoe; // =
        public TowerDef(string name, float range, float atk, float interval, bool aoe)
        {
            Name = name;
            Range = range;
            Atk = atk;
            Interval = interval;
            Aoe = aoe;
        }
    }
}
/// <summary>
/// P03 . V0.1— X009+IConfigService Design/configs/P03 JSON
/// 06 03 M0-M5 /L1 / / /
/// #C1272D x x x #4CAF50 A* S — AI
/// </summary>
public static class NewbieConfig
{
    public const string GameTitle = " ";
    public const string GameSlogan = " . ";
    public const int NewbieWaves = 4; // L1 4 W4= Boss
    public const int StartLives = 5; // -1 /
    public const float IntroSeconds = 3f; // M0
    public const float PrepStartSeconds = 15f; // 1
    public const int FreeTowerPlacements = 3; // x2+ x1
    public const float SpawnInterval = 0.8f;
}

```

```

public const float WaveGapSeconds = 2f;
public const float MaxTutorialSeconds = 210f; // ≤3.5min 06 ②
public const int KillCrystal = 2; // +2/ +8
public const int WaveClearCrystal = 8;
/// <summary> / </summary>;
public static readonly TowerDef GuanYu = new(" ", range: 280f, atk: 12f, interval: 0.8f, aoe: true);
public static readonly TowerDef HuangZhong = new(" ", range: 520f, atk: 26f, interval: 1.1f, aoe: false);
/// <summary>S 8 7 → </summary>;
public static readonly float[] PathX = { -460f, 340f, 340f, -340f, -340f, 340f, 340f, -500f };
public static readonly float[] PathY = { 740f, 740f, 300f, 300f, -140f, -140f, -580f, -580f };
/// <summary>8 3 = / / </summary>;
public static readonly float[] SlotX = { 0f, 0f, 0f, -320f, 320f, -320f, 320f, -420f };
public static readonly float[] SlotY = { 620f, 420f, -280f, 620f, 420f, -280f, 620f, -280f };
public const float BossHp = 900f;
public const float BossSpeed = 55f;
public const float QingLongHpRatio = 0.4f; // ≤40%→
public const float FocusSeconds = 4f; //
/// <summary>L1 W4=4 + </summary>;
public static (int Count, float Hp, float Speed) GetWave(int w) => w switch
{
    1 => (5, 26f, 75f),
    2 => (7, 32f, 78f),
    3 => (10, 38f, 82f),
    _ => (4, 40f, 86f),
};
/// <summary>06 M3 = ·id 0-5 BattleSim.PickTianji </summary>;
public static readonly (string Name, string Desc)[] TianjiCards =
{
    (" ", " -20% "),
    (" ", " +25% "),
    (" ", " +10"),
    (" ", " +15"),
    (" ", " +1"),
    (" ", " -15%"),
};
}
}
// OfflinePay.cs
using System;
namespace Sy.P03.Newbie
{
    /// <summary>
    /// P02 OfflineEarnings.T015-A5 3
    /// 8 =24/ 1
    /// BannerWallet
    /// 12 PlayerPrefs=
    /// </summary>;
    public static class OfflinePay
    {
        public const string SeenKey = "p03_offline_seen_ms";
    }
}

```

```
public const int CoinsPerHour = 3;
public const int MaxAccrualHours = 8;
public const int MaxPerDay = CoinsPerHour * MaxAccrualHours;
public const int MinClaimCoins = CoinsPerHour;
static readonly long MsPerHour = 3_600_000L;
public static int CoinsFor(long elapsedMs)
{
    if (elapsedMs < 0) return 0;
    long hours = elapsedMs / MsPerHour;
    if (hours > MaxAccrualHours) hours = MaxAccrualHours;
    return (int)hours * CoinsPerHour;
}
public static int CoinsBetween(long lastSeenMs, long nowMs)
    => CoinsFor(nowMs - lastSeenMs);
public static void Stamp(long nowMs)
{
    UnityEngine.PlayerPrefs.SetString(SeenKey, nowMs.ToString());
    UnityEngine.PlayerPrefs.Save();
}
public static void ClearForTests()
    => UnityEngine.PlayerPrefs.DeleteKey(SeenKey);
public sealed class OfflineClaim
{
    public int Pending { get; }
    public long SeenAtMs { get; }
    public bool Claimable => Pending >= MinClaimCoins;
    public bool Consumed { get; private set; }
    long _nowMs;
    OfflineClaim(int pending, long seenAtMs, long nowMs)
    {
        Pending = pending;
        SeenAtMs = seenAtMs;
        _nowMs = nowMs;
    }
    public static OfflineClaim Open(long nowMs)
    {
        long seen = 0;
        var raw = UnityEngine.PlayerPrefs.GetString(SeenKey, "");
        if (long.TryParse(raw, out var parsed)) seen = parsed;
        int pending = seen > 0 ? CoinsBetween(seen, nowMs) : 0;
        return new OfflineClaim(pending, seen, nowMs);
    }
    /// <summary> / </summary>
    public int TryCollect(bool doubled)
    {
        if (Consumed || Pending <= 0) return -1;
        int granted = doubled ? Pending * 2 : Pending;
        BannerWallet.Load().Deposit(granted);
        Stamp(_nowMs);
        Consumed = true;
    }
}
```

```
        return granted;
    }
}
}
}
// RunResult.cs
using System;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///     P02 RunResult.T006 —
    ///     / /
    /// </summary>
    public sealed class RunResult
    {
        public bool Won;
        public int Seed;
        public bool IsTianming;
        public bool IsTutorial;
        public int WaveReached;
        public int Kills;
        public int LifeLeft;
        public int CrystalsLeft;
        public int BannerEarned;
        public float DurationSec;
        public string FormationCode;
        public string CalamityName;
        public static RunResult Snapshot(BattleSim sim, bool won)
        {
            return new RunResult
            {
                Won = won,
                Seed = sim.Seed,
                IsTianming = sim.IsTianming,
                IsTutorial = sim.Profile == null,
                WaveReached = sim.Wave,
                Kills = sim.Kills,
                LifeLeft = sim.Lives,
                CrystalsLeft = sim.Crystals,
                DurationSec = sim.BattleTime,
                FormationCode = FormationCodec.Encode(sim),
                CalamityName = sim.Profile != null ? sim.Profile.CalamityName : "",
                BannerEarned = BannerWallet.EarnFor(won, sim.Wave, sim.Kills),
            };
        }
    }
}
// SiegeLevel.cs
using System;
namespace Sy.P03.Newbie
```

```

{
    /// <summary>
    /// V1.4          CEO   0-20261010-1245          —
    ///   =          /          P02 RandomLevel      X031
    ///   V1.4
    ///   NewbieConfig   =null profile          06 03
    ///   12   —          /
    /// </summary>
    public sealed class SiegeLevel
    {
        //   [5,8]   4   —
        //           seed=
        public const int WaveWindowMin = 5;
        public const int WaveWindowMax = 8;
        public readonly int Seed;
        public readonly int TotalWaves;
        public readonly float HpBase;          //          26
        public readonly float HpGrowth;        //
        public readonly float SpeedBase;        //          /
        public readonly float SpeedGrow;        //
        public readonly int CountBase;          //
        public readonly float SpawnInt;         //
        public readonly float WaveGap;          //
        public readonly int StartCrystals;      //
        public readonly bool MidBoss;           //
        public readonly string CalamityName;    //          HUD
        public readonly int CalamityId;         //          id BattleSim
        public readonly float CalamityPower;    //          0.1-0.25
        SiegeLevel(int seed, System.Random r, int waveMin, int waveMax)
        {
            Seed = seed;
            TotalWaves = r.Next(waveMin, waveMax + 1);
            HpBase = Lerp(r, 40f, 58f);
            HpGrowth = Lerp(r, 1.18f, 1.28f);
            SpeedBase = Lerp(r, 72f, 88f);
            SpeedGrow = Lerp(r, 1.5f, 3.5f);
            CountBase = r.Next(7, 10);
            SpawnInt = Lerp(r, 0.6f, 1.0f);
            WaveGap = Lerp(r, 1.6f, 2.6f);
            StartCrystals = r.Next(12, 20);
            MidBoss = r.NextDouble() < 0.5;
            CalamityId = r.Next(0, 5);
            CalamityPower = Lerp(r, 0.10f, 0.25f);
            CalamityName = CalamityId switch
            {
                0 => " ",
                1 => " ",
                2 => " ",
                3 => " ",
                _ => " ",
            }
        }
    }
}

```

```

    };
}
public static SiegeLevel Generate(int seed)
    => Generate(seed, WaveWindowMin, WaveWindowMax);
/// <summary>
    </summary>
public static SiegeLevel Generate(int seed, int waveMin, int waveMax)
    => new SiegeLevel(seed, new System.Random(seed), waveMin, waveMax);
/// <summary>
    NewbieConfig.GetWave
    BattleSim
    </summary>
public WaveDef GetWave(int w)
{
    float hp = HpBase * (float)Math.Pow(HpGrowth, w - 1)
        * (CalamityId == 1 ? 1f + CalamityPower : 1f);    //
    int count = CountBase + (w / 3) + (CalamityId == 2 ? 2 : 0);    //    +2
    float speed = SpeedBase + SpeedGrow * w + (CalamityId == 0 ? SpeedBase * CalamityPower : 0f);
    bool boss = w == TotalWaves || (MidBoss && w == TotalWaves - 1 && w >= 3);
    return new WaveDef(w, count, hp, speed, boss);
}
static float Lerp(System.Random r, float min, float max)
    => min + (float)r.NextDouble() * (max - min);
}
public readonly struct WaveDef
{
    public readonly int Wave;
    public readonly int Count;
    public readonly float Hp;
    public readonly float Speed;
    public readonly bool Boss;
    public WaveDef(int wave, int count, float hp, float speed, bool boss)
    {
        Wave = wave;
        Count = count;
        Hp = hp;
        Speed = speed;
        Boss = boss;
    }
}
}
// TianmingBoard.cs
using System;
using UnityEngine;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///
    P02 DestinyBoard.T013-B2. 12
    ///
    =    KV
    ///
    KV
    P02 DefenseChallenge.Beats
    /// > > —
    ///
    PlayerPrefs=
    /// </summary>
    public static class TianmingBoard

```

```
{
    public const string KvKey = "p03_tianming";
    const string BestCacheKey = "p03_tianming_best";
    const string DateTag = "\"date\":";
    const string SeedTag = "\"seed\":";
    const string WaveTag = "\"wave\":";
    const string KillsTag = "\"kills\":";
    const string WonTag = "\"won\":";
    public readonly struct TianmingScore
    {
        public readonly int Date;    // yyyymmdd
        public readonly int Seed;
        public readonly int Wave;
        public readonly int Kills;
        public readonly bool Won;
        public TianmingScore(int date, int seed, int wave, int kills, bool won)
        {
            Date = date;
            Seed = seed;
            Wave = wave;
            Kills = kills;
            Won = won;
        }
    }
}

public static int DateKeyFor(DateTime date)
    => date.Year * 10000 + date.Month * 100 + date.Day;
/// <summary>
/// </summary>
public static string PayloadFor(TianmingScore s)
    => "{" + DateTag + s.Date + "," + SeedTag + s.Seed + "," + WaveTag + s.Wave
        + "," + KillsTag + s.Kills + "," + WonTag + (s.Won ? 1 : 0) + "}";
public static bool TryParse(string payload, out TianmingScore score)
{
    score = default;
    if (string.IsNullOrEmpty(payload)) return false;
    if (!TryInt(payload, DateTag, out int date)) return false;
    if (!TryInt(payload, SeedTag, out int seed)) return false;
    if (!TryInt(payload, WaveTag, out int wave)) return false;
    if (!TryInt(payload, KillsTag, out int kills)) return false;
    if (!TryInt(payload, WonTag, out int won)) return false;
    score = new TianmingScore(date, seed, wave, kills, won == 1);
    return true;
}

public static bool IsBetter(TianmingScore candidate, TianmingScore current)
{
    if (candidate.Date != current.Date) return candidate.Date > current.Date;
    if (candidate.Won != current.Won) return candidate.Won;
    if (candidate.Wave != current.Wave) return candidate.Wave > current.Wave;
    return candidate.Kills > current.Kills;
}

/// <summary>
/// PlayerPrefs    KV
```

```
/// Boot = no-op </summary>
public static bool TrySubmit(RunResult result, Sy.Platform.IOpenDataBoard board)
{
    if (result == null || !result.IsTianming || board == null) return false;
    var candidate = new TianmingScore(
        DateKeyFor(DateTime.Now),
        result.Seed, result.WaveReached, result.Kills, result.Won);
    string cached = PlayerPrefs.GetString(BestCacheKey, "");
    if (TryParse(cached, out var best) && best.Date == candidate.Date
        && !IsBetter(candidate, best))
        return false;
    string payload = PayloadFor(candidate);
    PlayerPrefs.SetString(BestCacheKey, payload);
    board.SubmitScore(KvKey, payload);
    return true;
}
/// </summary> </summary>
public static bool TryGetLocalBest(out TianmingScore best)
    => TryParse(PlayerPrefs.GetString(BestCacheKey, ""), out best);
public static void ClearForTests() => PlayerPrefs.DeleteKey(BestCacheKey);
static bool TryInt(string payload, string tag, out int value)
{
    value = 0;
    int at = payload.IndexOf(tag, StringComparison.Ordinal);
    if (at < 0) return false;
    int start = at + tag.Length;
    int end = start;
    while (end < payload.Length && (char.IsDigit(payload[end])
        || (end == start && payload[end] == '-'))) end++;
    return end > start && int.TryParse(payload.Substring(start, end - start), out value);
}
}
}
// TianmingGate.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P03.Newbie
{
    /// </summary>
    /// P02 DestinyGate.T013-B
    /// 0..19 0= 5% = evt_tianming
    /// 12
    /// </summary>
    public static class TianmingGate
    {
        public const string RollKey = "p03_ab_tianming";
        public const int GrayDenominator = 20;
        public static int Roll()
        {
            int roll = PlayerPrefs.GetInt(RollKey, -1);
```

```

        if (roll < 0)
        {
            roll = UnityEngine.Random.Range(0, GrayDenominator);
            PlayerPrefs.SetInt(RollKey, roll);
        }
        return roll;
    }

    public static bool IsEligible() => Roll() == 0;
    public static string GroupLetter() => IsEligible() ? "b" : "a";
    public static void SetRollForTests(int roll) => PlayerPrefs.SetInt(RollKey, roll);
    public static void ClearForTests() => PlayerPrefs.DeleteKey(RollKey);
    /// <summary>evt_tianming / seed </summary>
    public static Dictionary<string, string> EntryFields(string action)
        => new Dictionary<string, string>
    {
        { "action", action },
        { "abGroup", GroupLetter() },
        { "seed", TianmingPin.SeedForToday().ToString() },
    };

    public static Dictionary<string, string> RunEndFields(int seed, bool won, int wave)
        => new Dictionary<string, string>
    {
        { "action", "run_end" },
        { "abGroup", GroupLetter() },
        { "seed", seed.ToString() },
        { "win", won ? "true" : "false" },
        { "wave", wave.ToString() },
    };
}

// TianmingPin.cs
using System;
namespace Sy.P03.Newbie
{
    /// <summary>
    /// P02 DestinyPin.T013-A —
    /// =
    /// 12 +
    /// </summary>
    public static class TianmingPin
    {
        /// <summary> → FNV-1a yyyyymmdd 8
        /// IL2CPP/wasm uint→int </summary>
        public static int SeedForDate(int year, int month, int day)
        {
            unchecked
            {
                uint h = 2166136261u;
                int date = year * 10000 + month * 100 + day;
                for (int i = 0; i < 8; i++)

```

```

        {
            uint digit = (uint)(date % 10);
            date /= 10;
            h = (h ^ digit) * 16777619u;
        }
        return (int)(h & 0x7FFFFFFFu);
    }
}

/// <summary>
/// </summary>
public static int SeedForToday()
{
    var now = DateTime.Now;
    return SeedForDate(now.Year, now.Month, now.Day);
}
}

// TutorialFlow.cs
using System;
namespace Sy.P03.Newbie
{
    /// <summary>
    ///     evt_newbie_step 1-5 ① ② ③ ④ Boss⑤ +
    ///     BattleSim / / / / —
    /// Play headless NewbieFlowCheck
    /// </summary>
    public sealed class TutorialFlow
    {
        public int Step { get; private set; }
        readonly Action<string> _hint;
        readonly Action<int> _onStep;
        bool _firstTowerSeen;
        public TutorialFlow(BattleSim sim, Action<string> hint, Action<int> onStep)
        {
            _hint = hint ?? delegate { };
            _onStep = onStep ?? delegate { };
            sim.TowerPlaced += OnTowerPlaced;
            sim.WaveCleared += OnWaveCleared;
            sim.TianjiOffered += OnTianjiOffered;
            sim.TianjiPicked += OnTianjiPicked;
            sim.BossKilled += OnBossKilled;
            sim.StateChanged += OnStateChanged;
            _hint(" —L1 x2 + x1 ");
        }
        void Advance(int step)
        {
            if (Step >= step) return;
            Step = step;
            _onStep(step);
        }
        void OnTowerPlaced(BattleSim.TowerInst tower, int slot)

```

```
{
    if (!_firstTowerSeen)
    {
        _firstTowerSeen = true;
        Advance(1);
        _hint("          —          ");
        return;
    }
    _hint(tower.IsGuanYu ? "          —          " : "          —          ");
}
void OnWaveCleared(int wave)
{
    if (wave == 1)
    {
        Advance(2);
        _hint("          —          ");
    }
}
void OnTianjiOffered()
{
    _hint("          —          -20%");
}
void OnTianjiPicked(int index)
{
    Advance(3);
    _hint(" " + NewbieConfig.TianjiCards[index].Name + "          " + NewbieConfig.TianjiCards[index].Desc + " —          ")
}
void OnBossKilled()
{
    Advance(4);
    _hint("          —L1          ");
}
void OnStateChanged(BattleSim.BattleState state)
{
    if (state == BattleSim.BattleState.Won)
    {
        Advance(5);
        _hint("L1          —          ");
    }
}
}
}
```